

# UML Basic

By Saturngod

## Introduction

UML ဟာ system တစ်ခု ကို ဖန်တီးသည့် အခါမှာ system အကြောင်းကို document အနေနဲ့ ပြန်ဖတ်သည့် အခါမှာ ဖြစ်ဖြစ် နားလည် လွယ်အောင် ရေးဆွဲထားသည့် ပုံတွေပါပဲ။ Design Pattern ကို လေ့လာ ဖို့ အတွက် UML ကို အရင် ဆုံး သိနေဖို့ လိုပါတယ်။ UML စာအုပ်ကို မဖတ်ခင်မှာ ဦးစွာ Object-oriented programming (OOP) ကို နားလည်နေ ဖို့ လိုပါတယ်။ OOP မှာ ပါသည့် Interface, Abstract, Inheritance တို့ကို သိနေမှသာ UML ကို လေ့လာလို့ ရပါလိမ့်မယ်။

## Drawing Tool

UML ကို ရေးဆွဲဖို့ Software တွေ အများကြီး ရှိပါတယ်။ <https://www.draw.io> ဟာ free ရပြီး လက်ရှိ စာအုပ်မှာ ဖော်ပြမည့် Use Case Diagram, Class Diagram, Sequence Diagram တွေကို ရေးဆွဲဖို့ အဆင်ပြေပါတယ်။

နောက်ပြီး UML ကို PlantUML ကို သုံးပြီး ရေးဆွဲနိုင်ပါတယ်။ <https://www.planttext.com/> မှာ စာသားဖြင့် ရေးသားပြီး UML class diagram, sequence diagram တွေကိုလည်း ရေးသားနိုင်ပါတယ်။

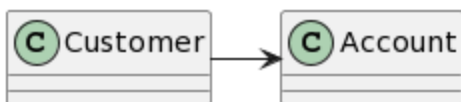
ဥပမာ

```
@startuml
class Customer {
}
class Account {
}

Customer -> Account

@enduml
```

အဲဒီလို Text အတွက် အောက်ပါ ပုံ အတိုင်း generate လုပ်ပေးပါတယ်။



Customer class ထဲမှာ Account class ကို ယူသုံးမယ်ဆိုပြီး One Way Associations လုပ်ထားသည့် သဘောပါ။ နောက်ပိုင်း Chapter တွေမှာ Associations ကို ရှင်းပြပါမယ်။

နောက်ထပ် Free ရသည့် Tool ကော့ StarUML ( <https://staruml.io/> ) ပါ။ လူသုံးများသည့် Software တစ်ခုပါ။ Export ထုတ်ရင်တော့ watermark ပါပါလိမ့်မယ်။

# Chapter 1 :: UML ဆိုတာ

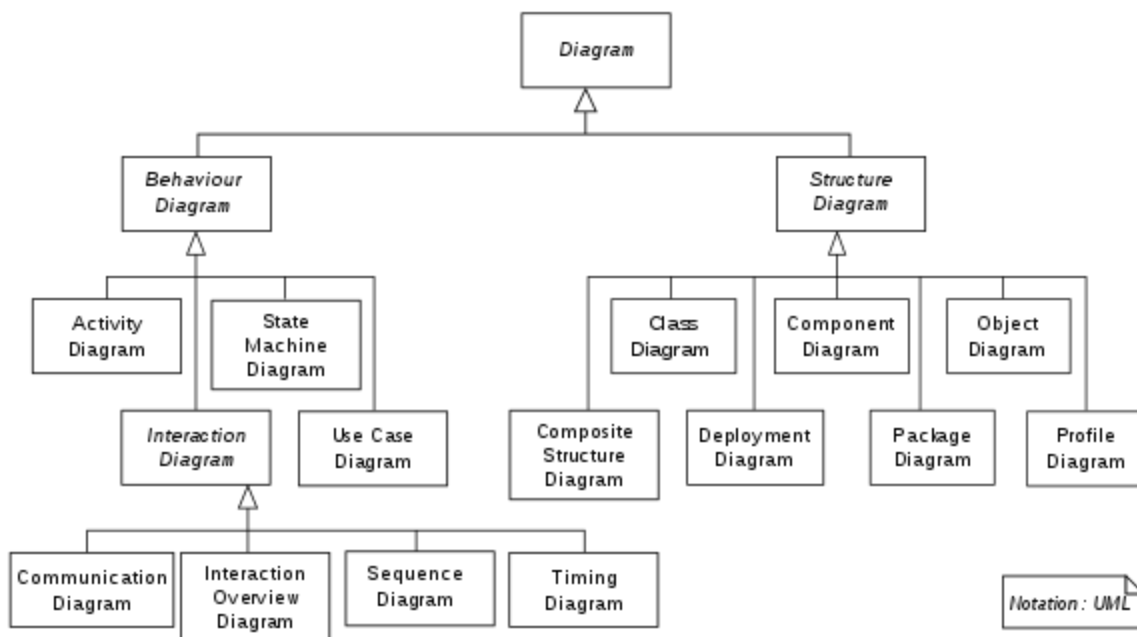
## UML

UML ဆိုတာက Unified Modeling Language ရဲ့ အတိုကောက်ဖြစ်ပြီး general purpose, developmental modeling language တစ်ခုပါ။ တနည်းပြောရင် software engineering အတွက် design system ကို မြင်အောင် ဖော်ပြရေးဆွဲသည့် language တစ်ခုပါ။ UML ဟာ လက်ရှိ version 2.5 အထိ ရှိသွားပြီ ဖြစ်ပါတယ်။

UML ကို Grady Booch, Ivar Jacobson, and James Rumbaugh တို့ သုံးယောက်ဟာ 1995 မှာ Rational Software တစ်ခုကို ဖန်တီးရင်း နဲ့ လုပ်ထားခဲ့တာပါ။ နောက်ပိုင်း 1997 မှာ Object Management Group ဟာ UML ကို standard အနေနဲ့ အသုံးပြုလာပါတယ်။ Object Management Group မှာ HP, IBM, Apple စသည့် company ကြီးတွေ ပါဝင်ပါတယ်။

## Diagram

UML ကို Diagram တွေ နဲ့ ရေးဆွဲပြီး UML မှာ ပါရှိသည့် diagram တွေကတော့ အောက်ပါ အတိုင်း ဖြစ်ပါတယ်။



ဒီစာအုပ်ထဲမှာတော့ အကုန်လုံး ဖော်ပြပေးနိုင်မှာ မဟုတ်ပါဘူး။ Software Development ပြုလုပ်သည့် အခါ လူသုံးများသည့်

- Use Case Diagram
- Sequence Diagram
- Class Diagram

စသည့် ၃ မျိုးကို ဖော်ပြပေးမှာပါ။

## Structure Diagrams

Structure diagrams ဟာ system မှာ ပါဝင်သည့် ပုံစံ ကို ဖော်ပြပေးထားတာပါ။ System တစ်ခုကို မတည်ဆောက်ခင် structure diagrams တစ်ခုခု ကို သုံးပြီး ဖန်တီးပါတယ်။ Software architecture document ပြုလုပ်ပေးသည့် အခါမှာ ထည့်သွင်း ဖော်ပြကြပါတယ်။ Class Diagram ဟာ structure diagram တစ်ခု ဖြစ်ပါတယ်။

## Behavior Diagrams

Behavior diagrams ကတော့ dynamic ဖြစ်ပါတယ်။ System မှာ ဘာတွေ ဖြစ်နေလဲ။ System မှာ ဘယ်လို function တွေ ပါပြီး ဘယ်သူတွေ က လုပ်ဆောင်နေလဲ ဆိုတာကို ဖော်ပြပေးပါတယ်။ Use Case Diagram ဟာ behavior diagrams အမျိုးအစား တစ်ခုပါ။

## Interaction Diagram

Interaction diagrams ဟာ system flow ကို ဖော်ပြပေးတာပါ။ Models တစ်ခု နဲ့ တစ်ခု ဘယ်လို အလုပ်နေတယ်။ Sequence Diagram ဟာ interaction diagram တစ်ခု ဖြစ်ပြီး system တစ်ခု လုံး ဘယ်လို အလုပ်လုပ်တယ် ဆိုတာကို အဆင့်ဆင့် ဖော်ပြပေးထားပါတယ်။

# Chapter 2 :: Use Case Diagram

System တစ်ခု စတင်ဖို့ အတွက် Use Case Diagram ဟာ အရေးကြီးပါတယ်။ Use Case Diagram ကို မစတင်ခင်မှာ Use Case တွေကို အရင်ဆုံး စဉ်းစားဖို့ လိုအပ်ပါတယ်။

## Use Case

ATM system မှာ ငွေထုတ် ဖို့ အတွက် Use Case တွေကို ကြည့်ရအောင်။

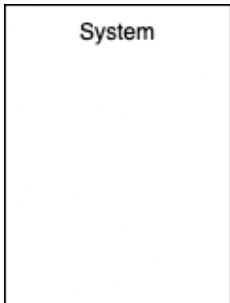
| User Action              | System Response                                         |
|--------------------------|---------------------------------------------------------|
| User မှ Pin ရိုက်ထည့်    | System မှ PIN မှန် မမှန် စစ်ဆေး                         |
| Withdraw Amount ကို ရွေး | System မှ Withdraw Amount လုပ်နိုင်သည့် ပမာဏ ကို ဖော်ပြ |
| ငွေ Amount ရိုက်ထည့်     | ငွေ Amount ရှိမရှိ စစ်ဆေး ပြီး ငွေထုတ်ပေး               |

Use Case ဟာ System တစ်ခု အတွက် အရေးပါပြီး System တစ်ခုလုံးမှာ လိုအပ်သည့် requirement တွေကို တစ်ခါတည်း မြင်နိုင်ပါလိမ့်မယ်။

## Use Case Diagram

Use Case Diagram မှာ ပါဝင်သည့် diagram တွေ ကို အရင်လေ့လာကြည့်ရအောင်။

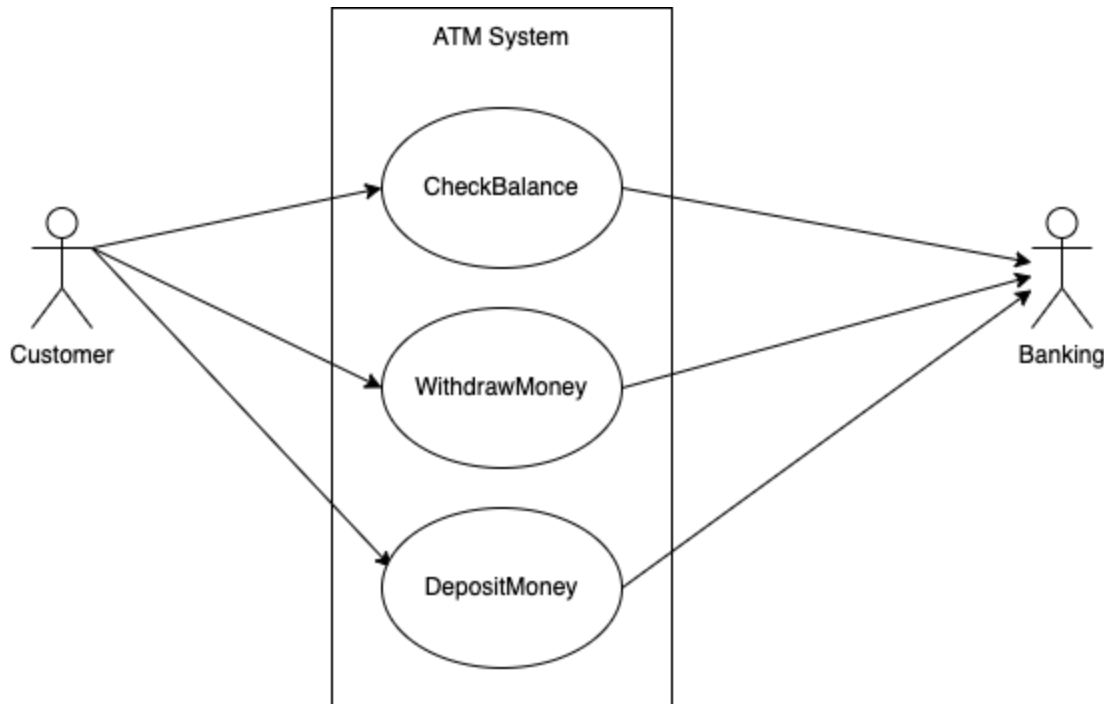
| Diagram                                                                                         | Description                                                              |
|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <br>Actor    | Actor ကတော့ အသုံးပြုမည့်သူတွေ လုပ်ဆောင်မည့် သူတွေ ကို ရည်ညွှန်းထားပါတယ်။ |
| <br>Use Case | Use Case ကတော့ Functional case တွေကို ဖော်ပြထားပါတယ်။                    |

| Diagram                                                                           | Description                                                                                               |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
|  | <p>System ကတော့ လက်ရှိ system ကြီး တစ်ခုလုံးကို ဘယ် system ဖြစ်တယ် ဆိုတာကို ဖော်ပြထားခြင်း ဖြစ်ပါတယ်။</p> |

## ATM System

ATM System တစ်ခုအတွက် Use Case တစ်ခု ရေးဆွဲကြည့်ရအောင်။

ATM System တစ်ခုမှာ Balance စစ်ခြင်း, ပိုက်ဆံ ထုတ်ခြင်း နှင့် ပိုက်ဆံ ထည့်သွင်းခြင်း တို့ ပါဝင်ပါမယ်။



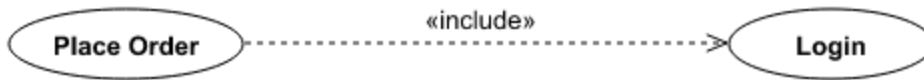
System ဖြစ်သည့် recent box ကို ထည့်သွင်းမရေးဆွဲလည်း ဖြစ်ပါတယ်။ System တစ်ခု ထက်ပိုမှသာ ထည့်သွင်း ရေးဆွဲလို့ရပါတယ်။

## Use Case Relationship

Use Case diagram မှာ `<<include>>` , `<<extend>>` နဲ့ Generalization ဆိုတာ ရှိပါတယ်။ Use Case တစ်ခုက နောက်ထပ် Use case တစ်ခု နဲ့ ဘယ်လို relationship ရှိနေတယ် ဆိုတာ ကို ဖော်ပြပေးတာပါ။



Invalid Password ဟာ Login Account ပေါ်မှာ depend ဖြစ်နေပါတယ်။ နောက်ပြီး Invalid Password ဟာ Login Account Use Case ကို extend လုပ်ထားတာပါ။



Place Order လုပ်ဖို့ အတွက် အရင်ဆုံး Login ဝင်ဖို့ လိုပါတယ်။ Place Order Use case မှာ Login User Case ပါဝင်နေပါတယ်။



generalization relationship က တော့ parent နဲ့ child သဘောပါပဲ။ Use case နှစ်ခုဟာ parent နဲ့ child relationship ရှိနေသည့် သဘောပါပဲ။

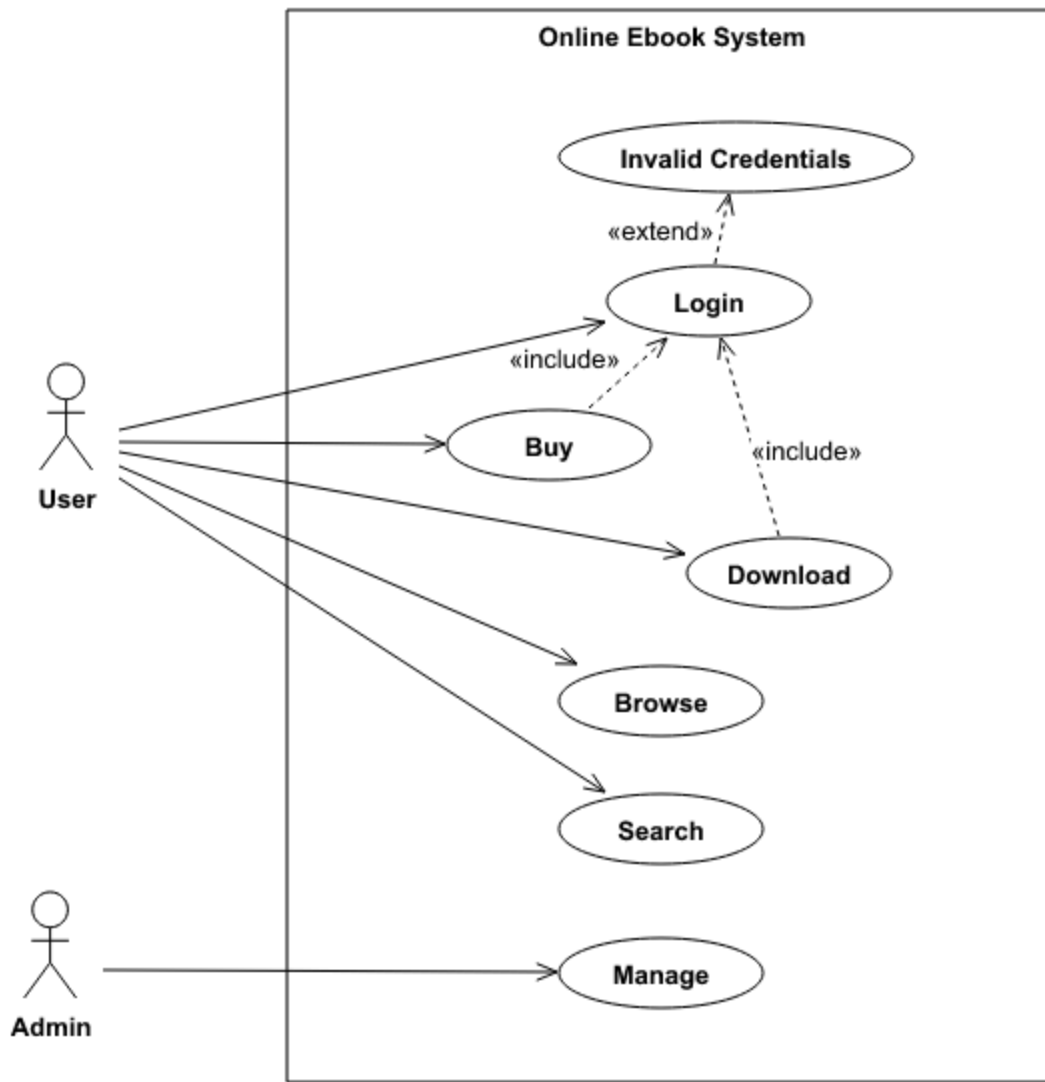
## Online Ebook Website System

System တစ်ခု ဆွဲကြည့်ရအောင်

- User က စာအုပ်တွေကို Search လုပ်နိုင်တယ်
- User က စာအုပ်တွေကို browse လုပ်နိုင်တယ်
- User က Login ဝင်လို့ရတယ်
- Login ဝင်သည့် အခါမှာ password မှားနေရင် မှားနေကြောင်း ပြရမယ်
- စာအုပ်ဝယ်သည့် အခါမှာ Login ဝင်ထားပြီးသား ဖြစ်ရမယ်
- Download ချသည့် အခါမှာလည်း Login ဝင်ထားဖို့ လိုတယ်
- Admin က စာအုပ်တွေကို manage လုပ်နိုင်မယ်

ဒီ စနစ်ကို ကြည့်လိုက်ရင် Actor ၂ ယောက် ပါတာကို တွေ့နိုင်ပါတယ်။ Use Case တွေက တော့ ရှင်းပါတယ်။ ပုံလေး ဆွဲကြည့်ရအောင်။





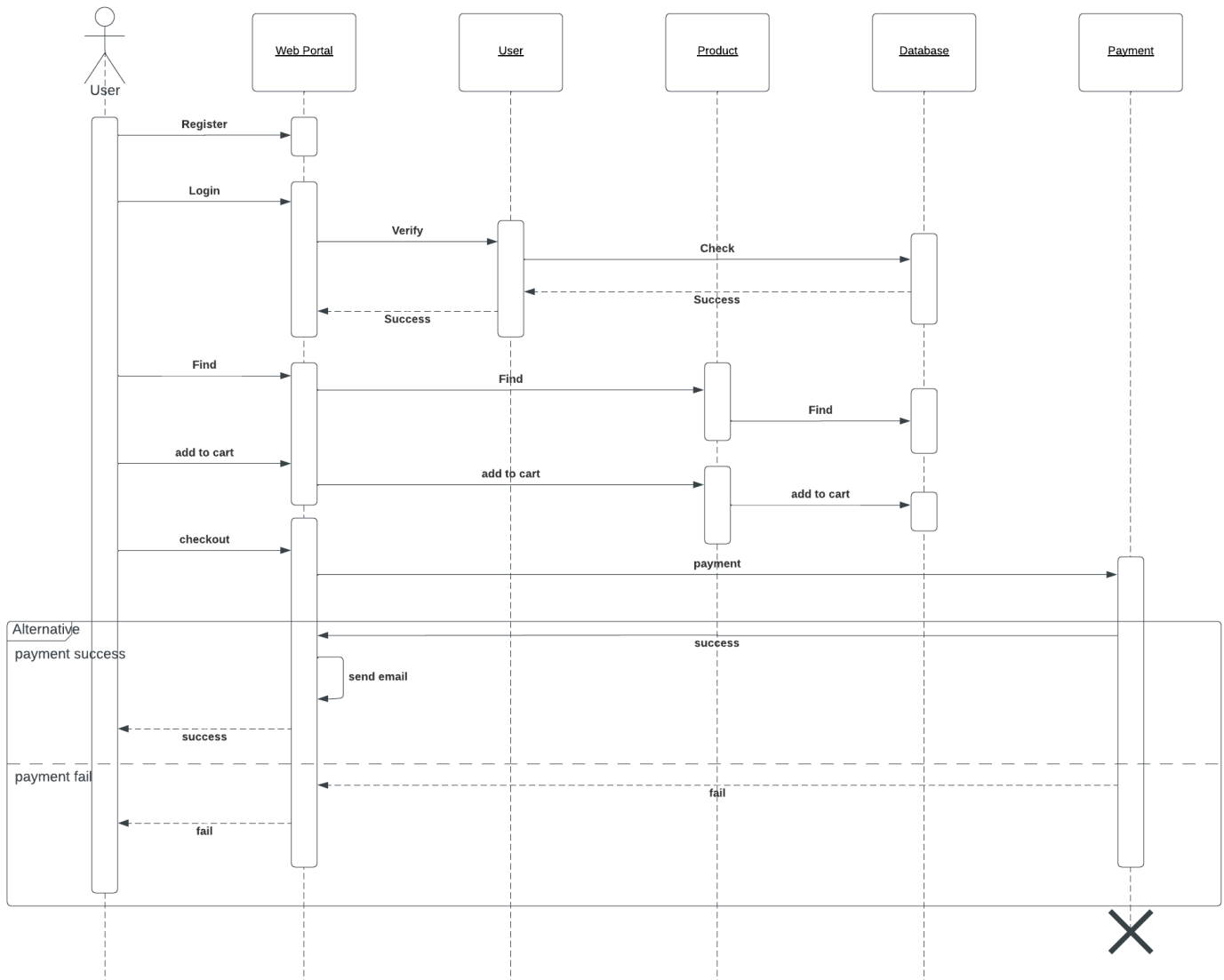
Use Case Diagram ဆွဲထားခြင်း အားဖြင့် System ကို ပိုပြီး နားလည် စေပါတယ်။ Project တစ်ခုလုံးမှာ ဘယ် system တွေက ဘယ် system တွေ နဲ့ ချိတ်ဆက်ထားတယ်။ ဘယ်လို depen ဖြစ်နေတယ် ဆိုတာကို သိနိုင်ပါတယ်။

# Chapter 3 :: Sequence Diagram

Sequence Diagram ဟာ UML မှာ အရေးပါသလို Project တွေ ဖန်တီးရာမှာလည်း အရေးပါပါတယ်။ Sequence Diagram ဟာ system တစ်ခု မှာ ဖြစ်သွားသည့် အဆင့်ဆင့် ကို ရေးဆွဲဖော်ပြထားခြင်း ဖြစ်ပါတယ်။ System မှာ ဘယ် အဆင့်တွေ အလုပ်လုပ်သွားတယ်။ ဘယ် အဆင့်ပြီးရင် ဘယ်ကို ခေါ်သလဲ ဆိုတာ ကို sequence diagram မှာ မြင်နိုင်ပါတယ်။ သူက behavioral diagram တစ်ခု ဖြစ်ပါတယ်။

## Sequence Diagram

Sequence Diagram ပုံကြမ်း တစ်ခုကို ကြည့်လိုက်ပါ။ ပုံကို ကြည့်လိုက်တာ နဲ့ sequence diagram ဆိုတာ ဘာလဲ ဆိုတာ သဘောပေါက်သွားမှာပါ။

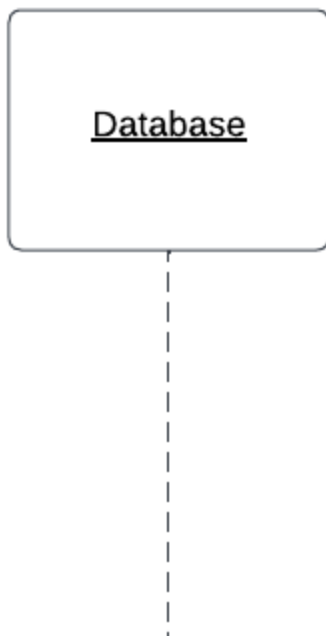


## Actor



Sequence Diagram မှာ Use Case လိုပဲ လုပ်ဆောင်သည့်သူ Actor ပါဝင်ပါတယ်။ External Subject တွေကိုလည်း actor အနေနဲ့ သတ်မှတ်လို့ရပါတယ်။

## Lifeline



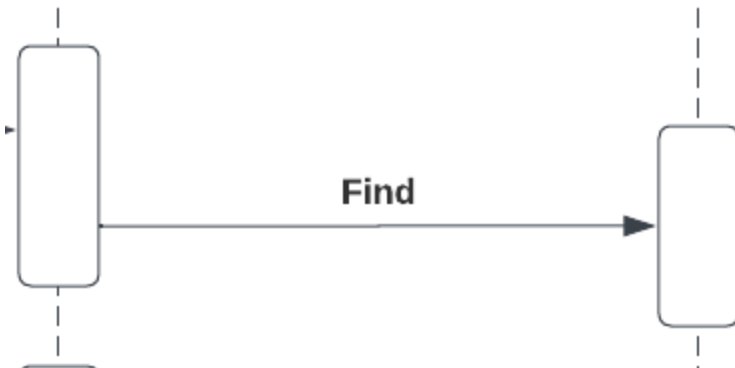
Lifeline က interaction တစ်ခုကို ဖော်ပြထားတာပါ။ ဘာ လုပ်မယ်။ အချိန် ဘယ်လောက်ကြာမယ်။ ဘယ်အချိန်မှာ ရပ်သွားမယ်။ စတာတွေကို Lifeline မှာ ဖော်ပြထားပါတယ်။

## Activations



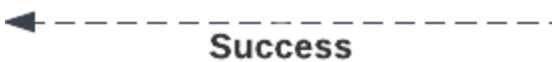
Activations ကတော့ Lifeline မှာ ဆွဲထားသည့် လေးထောင့် အတုံးလေး ဖြစ်ပြီး operation ရဲ့ အချိန် ကို ဖော်ပြထားတာပါ။

## Call Message



Activations တစ်ခု နဲ့ တစ်ခု ကြားက interaction ကို ဖော်ပြသည့် ဖော်ပြချက်ပါ။ ဘာ လုပ် လိုက်သလဲ ဆိုတာကို ဖော်ပြထားတာ မျိုးပါ။

## Return Message



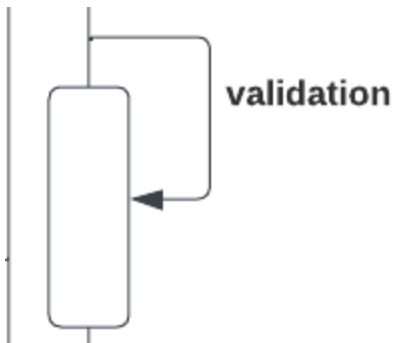
Operation ပြီးသွားလို့ ပြန်လာသည့် return message ပါ။ Function တစ်ခုကနေ return ပြန်လိုက်သလိုမျိုး ကို ဖော်ပြပေးပါတယ်။

## Self Message



Self Message ဆိုတာကတော့ ကိုယ့် Lifeline ကို ကိုယ် ပြန်ခေါ်သည့် သဘောပါ။

## Recursive Message



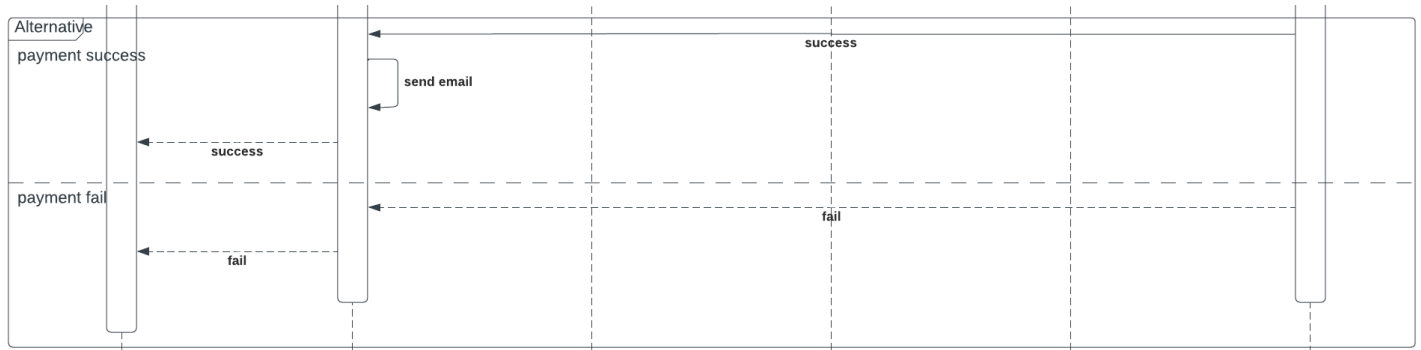
ကိုယ့် Lifeline မှာ ကိုယ် ပြန်ခေါ်ပြီးတော့ operation process time ရှိနေသည့် သဘောကို ဖော်ပြခြင်းသည့် အခါမှာ ရေးဆွဲပါတယ်။

## Stopped



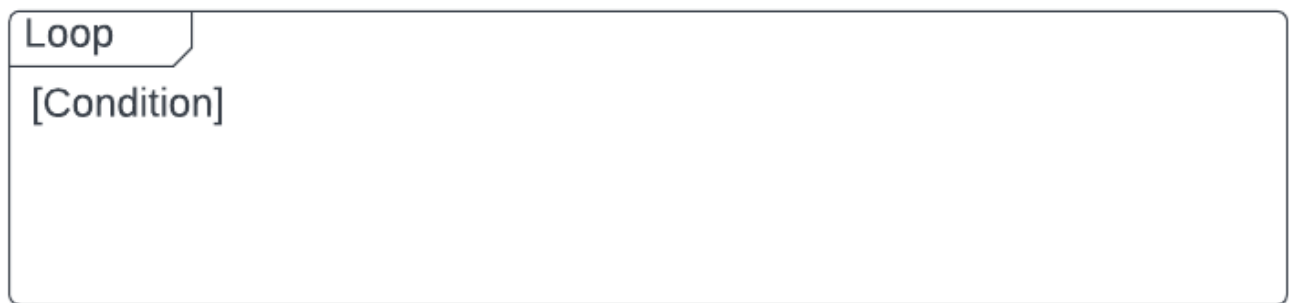
Lifeline တစ်ခု ပြီး ဆုံး တာကို ဖော်ပြထားခြင်း ဖြစ်ပါတယ်။

## Alternative



Alternative ကတော့ condition တစ်ခုခု success ဖြစ်ခဲ့ရင် ဘယ် flow ကို သွားမယ်။ fail ဖြစ်ရင် ဘယ်လို flow သွားမယ် ဆိုပြီး ပိုင်းခြားပြီး ရေးဆွဲထားခြင်းဖြစ်ပါတယ်။

## Loop



Loop လည်း အတူတူပါပဲ။ Condition တစ်ခု အထိ သွားမယ် ဆိုပြီး ထည့်သွင်းရေးဆွဲခြင်းပါ။

## ဘကြောင့် သုံးသင့်လဲ

Sequence Diagram က system design တစ်ခုကို ရှင်းပြသည့် အခါမှာ မဖြစ်မနေ အသုံးဝင်ပါတယ်။ ဥပမာ payment gateway system တစ်ခု ဆိုပါအို့။ User က ဘယ်လို payment လုပ်သွားတယ်။ ဘယ်အဆင့်တွေကို ခေါ်ပြီး server ကို data တွေ ဘယ်အချိန်မှာ ပြန်ရောက်လာမယ် ဆိုတာကို စာတွေ့ရှင်းပြနေခြင်းထက် sequence diagram ရေးဆွဲပြီး ရှင်းပြသည့် အခါမှာ နားလည် လွယ်ပါလိမ့်မယ်။

Sequence Diagram က မခက်ခဲ သလို system အစိတ်အပိုင်း ခြင်း တစ်ခု ဆီကို အဆင့်တိုင်း အဆင့် တိုင်း ဖော်ပြပေးနိုင်ပါတယ်။

# Chapter 4 :: Class Diagram

Class Diagram တစ်ခု ဖန်တီးဖို့ အတွက် System အကြောင်းကို ဦးစွာ နားလည်ဖို့ လိုပါတယ်။ လက်ရှိ System က ဘာကို ဖန်တီး ချင်တာလဲ။ ဘာတွေ လိုအပ်တယ်ဆိုတာကို ဦးစွာ Analysis လုပ်ရပါမယ်။ ပြီးသည့် အခါမှာ လိုအပ်သည့် class တွေကို ခွဲထုတ်ပြီး class diagram ကို ဖန်တီးပါတယ်။ Class တစ်ခု နဲ့ တစ်ခု realtion တွေကို ဖော်ပြသည့် အခါမှာ

- Association
- Dependency
- Inheritance

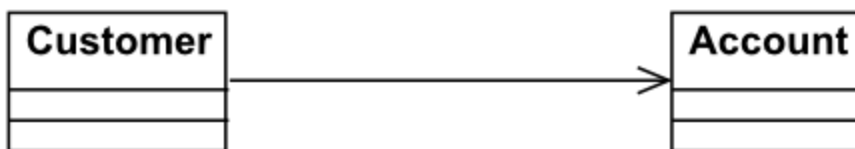
စသည် တို့ နဲ့ ဖော်ပြပါတယ်။

## Associations

Associations ဆိုတာက class တစ်ခု က နောက်တစ်ခုကို အသုံးပြုထားတယ် ဆိုတာကို ဖော်ပြထားခြင်း ဖြစ်ပါတယ်။

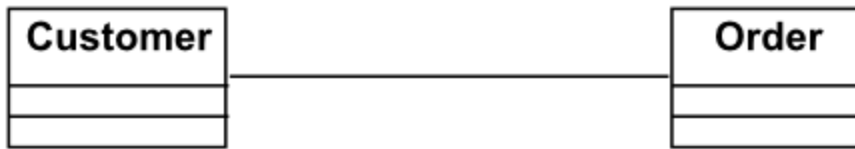
```
public class Customer {
    private Account checkingAccount;
    public void createNewAccount() {
        checkingAccount = new Account();
    }
}
```

ဒီ code မှာ Customer class မှာ Account class ကို ယူသုံးထားပါတယ်။ အဲဒီလိုမျိုး code အတွက် class ကို အောက်ပါ အတိုင်း ဆွဲပါတယ်။



**One-way associations** ဖြစ်သည့် အတွက် Arrow နဲ့ သုံးထားတာကို တွေ့နိုင်ပါတယ်။ A to B သဘောမျိုး ပါ။

အကယ်၍ A to B , B to A relation ရှိနေပြီ ဆိုရင်တော့ Arrow မပါတော့ပါဘူး။ ဥပမာ Customer class က Order ကို သုံးမယ်။ Order ထဲမှာလည်း Customer ပါတယ်။ ဒါဆိုရင် **Two-way associations** ဖြစ်ပါပြီ။



## Attributes

Class diagram မှာ properties တွေ ကို Access Modifier ပေါ်မှာ မှုတည်ပြီး အနည်းငယ် ပြောင်းလဲဖော်ပြပါတယ်။

```

public class Customer {
    public String name;
    private String city;
    protected String address;
    private Account account;

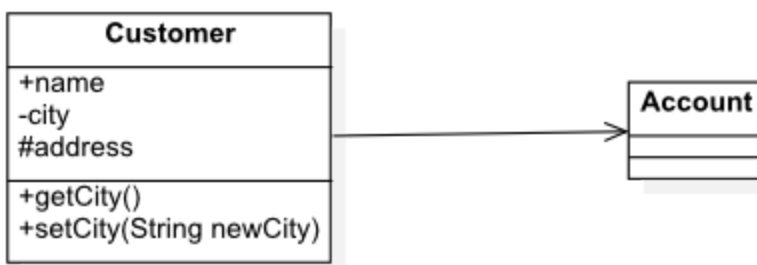
    public String getCity() {
        return city;
    }

    public void setCity(String newCity) {
        city = newCity;
    }
}
  
```

အဲဒီ အခါ class diagram ကို အောက်ပါ အတိုင်း ဆွဲပါတယ်။

- public ကို +
- private ကို -
- protected ကို #

အနေနဲ့ သုံးပါတယ်။ Account ကတော့ class ဖြစ်သည့် အတွက် direct associations ကို သုံးပါတယ်။

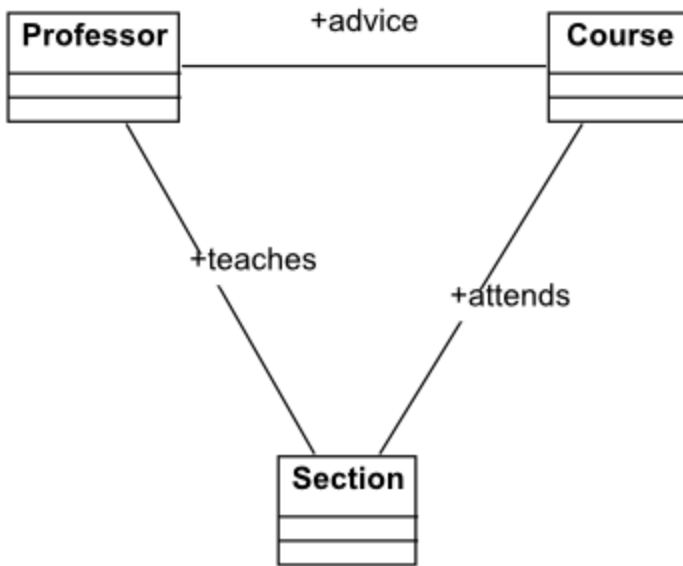


Association ရဲ့ နာမည်တွေဟာ **Verb** ဖြစ်နေဖို့လိုပါတယ်။ ဥပမာ

- Professor **advises** a Student
- Library **has** books



- Student **enrolls** in a Class

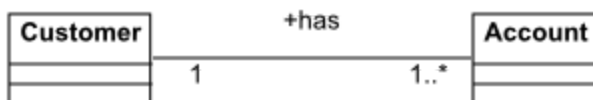


## Association Multiplicities

Association မှာ တစ်ခု နဲ့ တစ်ခု ပါဝင် ပတ်သက်မှုကို ဖော်ပြပေးသည့် Multiplicities ဆိုတာ ရှိပါသေးတယ်။

| Number | Description             |
|--------|-------------------------|
| 1      | One (mandatory)         |
| 3      | Three (exactly)         |
| *      | Many                    |
| 0..*   | zero or more (optional) |
| 1..*   | one or more             |
| 0..1   | zero or one (optional)  |

Multiplicities ကို အောက်ပါ အတိုင်း class တွေပါ အသုံးပြုပါတယ်။



ဒီ diagram မှာ ဆိုရင် Multiplicities ရဲ့ အဓိပ္ပာယ်က

- Customer တစ်ယောက်မှာ account ၁ ခု သို့မဟုတ် တစ်ခု ထက် မက ရှိတယ်။
- Account တစ်ခုဟာ Customer တစ်ယောက် အတွက်ပဲ။

Code အရ ကြည့်မယ် ဆိုရင်

```

public class Customer {
    private String name;
    private List<Account> accounts;

    public Customer(String name) {
        this.account = new ArrayList<Account>();
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public String addAccount(Account account) {
        this.accounts.add(account);
    }
}

```

```

public class Account {
    private String accountNumber;
    private double balance;
    private Customer customer;

    public Account(String accountNumber) {
        this.accountNumber = accountNumber;
    }

    public void setCustomer(Customer customer) {
        this.customer = customer;
    }

    public Customer getCustomer() {
        return this.customer;
    }

    public String getAccountNumber() {
        return this.accountNumber;
    }

    public double getBalance() {
        return this.balance;
    }
}

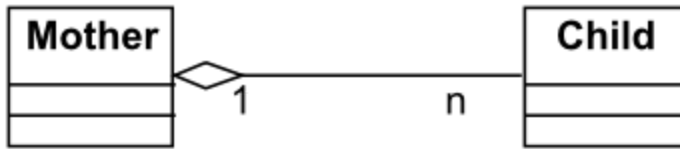
```

## Aggregation

Aggregation က **whole-part** relationship အတွက် အသုံးပြုပါတယ်။ Association မှာ **contains** အနေနဲ့ ပြောခဲ့ပြီးပါပြီ။ Aggregation အနေနဲ့ code အနေနဲ့ အတူတူပါပဲ။ Aggregation က ဘယ်လို နေရာမှာ အသုံးပြုလဲ ဆိုတော့

```
class Child {}
class Mother {
    List<Child> children;
}
```

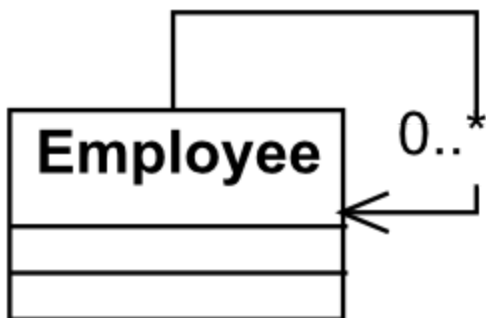
Class Diagram အရ အောက်ပါအတိုင်း ဖော်ပြပါတယ်။



တနည်းအားဖြင့် relation က အရမ်း strong ဖြစ်တယ်။ Child class ဟာ Mother class မရှိပဲ မရပ်တည်နိုင် ပါဘူး။ အဲဒီလို case တွေမှာ ဆိုရင်တော့ Aggregation ကို အသုံးပြုနိုင်ပါတယ်။

## Reflexive Association

Relationship ဟာ same class မှာလည်း အခြင်းခြင်း ဖြစ်နိုင်ပါတယ်။ ဥပမာ ဝန်ထမ်း တွေ အောက်ပါ ဝန်ထမ်းတွေ ရှိနိုင်ပါသေးတယ်။

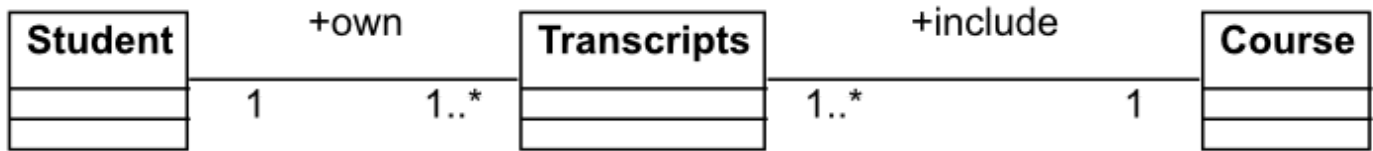


Code အရ ဆိုရင်

```
class Employee {
    List<Employee> subordinates;
}
```

## Association Classes

တစ်ခါတစ်လေ class နှစ်ခု ကြားက relation က အခြား class တစ်ခု နဲ့ ချိတ်ဆက်ထားတာ မျိုးရှိပါတယ်။



Student နဲ့ Course က relation ကို Transcripts ကနေ တစ်ဆင့် ရှိပါတယ်။ Code ကို ကြည့်ရအောင်။

```

class Student {
    Set<Transcript> transcripts = new HashSet<>();
}

class Course {
    Set<Transcript> transcripts = new HashSet<>();
}

class Transcript {
    Student student;
    Course course;
    Date subscriptionDate;

    public Transcript(Student student, Course course, Date subscriptionDate) {
        this.student = student;
        this.course = course;
        this.subscriptionDate = subscriptionDate;
        student.transcripts.add(this);
        course.transcripts.add(this);
    }

    public Course getCourse() {
        return this.course;
    }
}
  
```

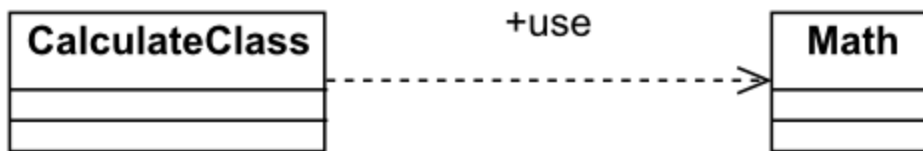
## Dependency

Dependency က တော့ Class တစ်ခုက တစ်ခြား Class တစ်ခုပေါ်မှာ depend လုပ်ထားပါတယ်။ ဥပမာ CoffeeApp က CoffeeMachine ပေါ်မှာ depend ဖြစ်ပါတယ်။ ဒါပေမယ့် CoffeeApp က ကြိုက်သည့် CoffeeMachine ကို ပြောင်းသုံးလို့ရတယ်။ Associate နဲ့ မတူတော့ကတော့ သူက strong relation မရှိပါဘူး။ တစ်ခြားဟာ တစ်ခု ပြောင်းသုံးပြီးလည်း class က အသုံးပြုလို့ရနေသေးတာပါ။ ဥပမာ

```
public class CalculateClass {
    public double calculateSquareRoot(double number) {
        return Math.sqrt(number);
    }

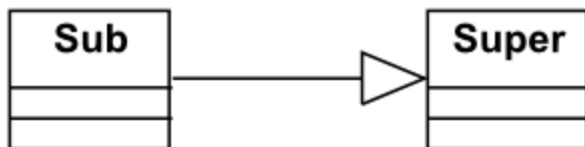
    public double calculatePower(double base, double exponent) {
        return Math.pow(base, exponent);
    }
}
```

CalculateClass က Math class အပေါ်မှာ dependency ရှိနေတာပါ။



## Inheritance

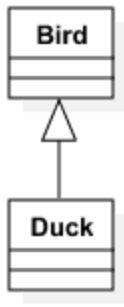
UML class diagram မှာ inheritance တွေကို အောက်ကလို မျိုး ကြိမ် ပုံ နဲ့ ဖော်ပြပါတယ်။



Code နဲ့ တွဲ ကြည့်ရအောင်။

```
public class Bird {
}
public class Duck extends Bird {
}
```

အထက်ပါ class လိုမျိုးကို အောက်ကလို diagram ဆွဲနိုင်ပါတယ်။



Inheritance ကို သုံးသည့် နေရာတွေ အများကြီး တွေ့နိုင်ပါတယ်။ Manager is Employee ဆိုသည့် relationship တွေ ရှိသည့် နေရာတွေမှာ Inheritance ကို အသုံးပြုနိုင်ပါတယ်။

သတိထားရမှာက Relationship မရှိရင် Inheritance မသုံးသင့်ပါဘူး။

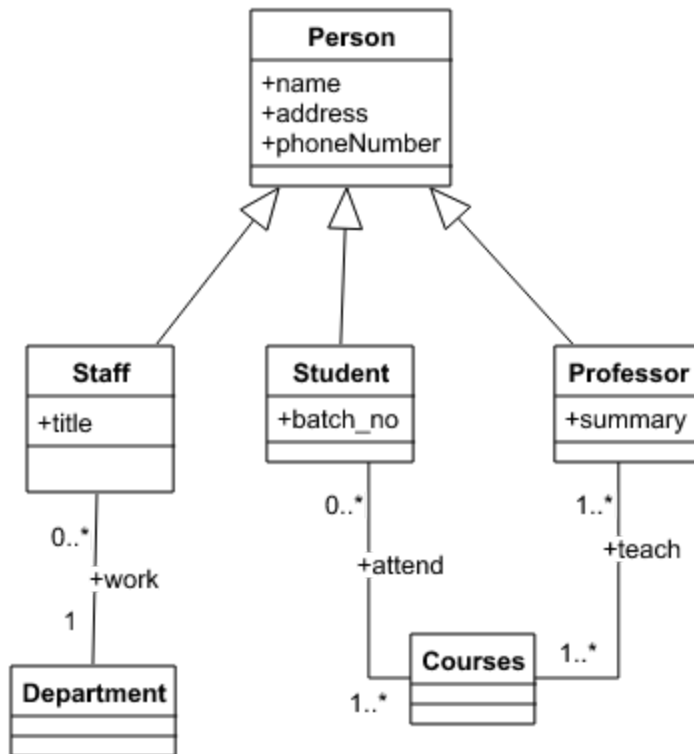
ဥပမာ။ ။ Car နဲ့ Bicycle လိုမျိုးပေါ့။ ကားမှာလည်း gear ပါတယ်။ Bicycle မှာလည်း gear ပါတယ်။ Car မှာလည်း အရောင်ပါတယ်။ model ပါတယ်။ Bicycle လည်း အတူတူပဲ။ properties တွေ တူသလို အချို့ basic function တွေလည်း တူတယ် ဆိုပြီး Inheritance ကို အသုံးပြုလို့ မဖြစ်ပါဘူး။

နောက်ပြီးတော့ Code Reuse တစ်ခု တည်း အတွက်လည်း မသုံးသင့်ပါဘူး။ Relationship မရှိသည့် School , Professor, Student မှာ ပါသည့် properties တွေ တူတယ် ဆိုပြီး Nameable class ဆိုပြီး ခွဲမထုတ်သင့်ပါဘူး။

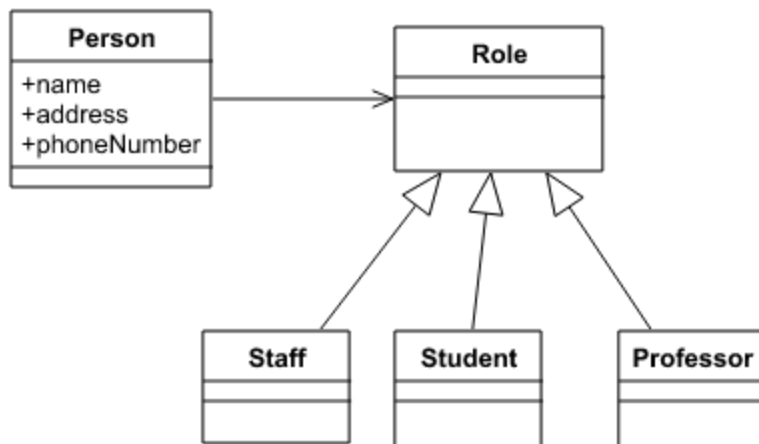
## Composition

Class နှစ် ခု က IS-A Relation မရှိဘူး။ ဒါပေမယ့် တစ်ခု class နှစ်ခု လုံးက လိုအပ်တယ်။ အဲဒီလို case တွေ ကို Inheritance အစား composition ပြောင်းသုံးသင့်ပါတယ်။

ဥပမာ အောက်ပါ class ကို ကြည့်ပါ။



ဒီမှာ ဆိုရင် Person ဆိုသည့် class ကို Inheritance ကို အသုံးပြုထားတာ တွေ့နိုင်ပါတယ်။ တကယ်တမ်းက Staff, Student, Professor တို့ဟာ Person ဖြစ်ပေမယ့် Role ပဲ မတူညီကြတာပါ။ Class diagram တွေကို Person ကို Inheritance သုံးမယ့် အစား Composition ပြောင်းသုံးတာ ပိုအဆင်ပြေပါလိမ့်မယ်။



Person object က Role ကို Composition အနေနဲ့ သုံးထားတယ်။ Staff, Student, Professor က Role ကို Inheritance လုပ်ထားတယ်။ ဒါကြောင့် Person နဲ့ Role နဲ့ ကွဲသွားပါတယ်။

```
public class Person {
    private String name;
    private String address;
    private String phoneNumber;
    private Role role; // Reference to Role

    // getters and setters
}

public abstract class Role {
    // Role specific attributes and methods
}

public class Staff extends Role {
    // Staff specific attributes and methods
}

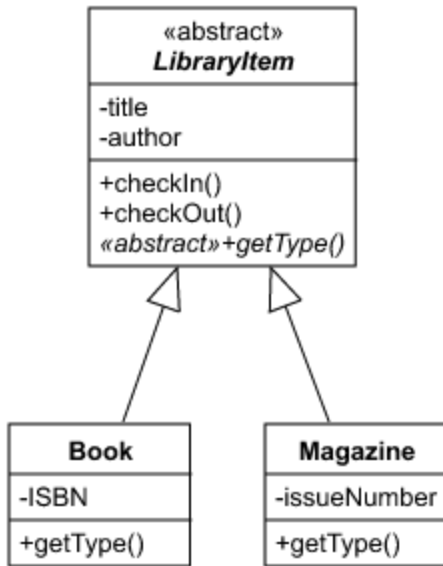
public class Student extends Role {
    // Student specific attributes and methods
}

public class Professor extends Role {
    // Professor specific attributes and methods
}
```

## Abstract and Interface

OOP မှာ abstract နဲ့ Interface က အရေးကြီးပါတယ်။ အကြမ်းအားဖြင့် abstract နဲ့ interface ကွာခြားချက်ကတော့ abstract က abstract data တွေ အပြင် function တွေပါ ထည့်သွင်းရေးသားနိုင်ပါတယ်။ interface က declaration အတွက်ပဲ အသုံးပြုနိုင်ပါတယ်။ ဘယ်အချိန်မှာ abstract, ဘယ် အချိန်မှာ interface တွေ သုံးသင့်သလဲ ဆိုတာကို design pattern ကို လေ့လာသည့် အခါမှာ တွေ့ရပါလိမ့်မယ်။





ဒီ Diagram မှာ ဆိုရင် abstract class ကို သုံးထားပါတယ်။ **checkIn** နဲ့ **checkOut** က **LibraryItem** ထဲမှာ အလုပ်လုပ်မှာ ဖြစ်ပြီး **getType** ကတော့ abstract function ဖြစ်သည့် အတွက် **Book** နဲ့ **Magazine** မှာ တွေ့နိုင်ပါတယ်။ Abstract ရဲ့ connection က Inheritance နဲ့ အတူတူပါပဲ။ Code အနေနဲ့ ကြည့်ရအောင်။

```

public abstract class LibraryItem {
    private String title;
    private String author;
    private boolean checkOut;

    public abstract String getType();
}

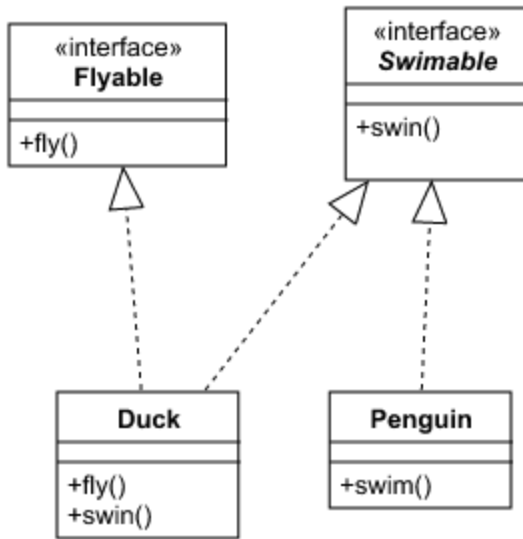
public class Book extends LibraryItem {
    private String ISBN;

    @Override
    public String getType() {
        return "Book";
    }
}

public class Magazine extends LibraryItem {
    private int issueNumber;

    @Override
    public String getType() {
        return "Magazine";
    }
}
  
```

Interface ကို ကြည့်ရအောင်။



interface ရဲ့ connection က inheritance နဲ့ မတူပဲ နည်းနည်း ကွာပါတယ်။ dotted ကို သုံးပြီး ရေးဆွဲထားပါတယ်။ ဒီ diagram မှာ Duck က Flyable, Swimable ကို သုံးထားပြီး Penguin ကတော့ Swimable ကိုပဲ အသုံးပြုထားတာကို တွေ့နိုင်ပါတယ်။ Interface က inheritance နဲ့ မတူတာက multiple implement လုပ်လို့ရပါတယ်။ Code ကို ကြည့်ရအောင်။

```

public interface Flyable {
    void fly();
}

public interface Swimable {
    void swim();
}

public class Duck implements Flyable, Swimable {
    @Override
    public void fly() {
        // Implement fly behavior for Duck
    }

    @Override
    public void swim() {
        // Implement swim behavior for Duck
    }
}

public class Penguin implements Swimable {
    @Override
    public void swim() {
        // Implement swim behavior for Penguin
    }
}
  
```

## Exercise

Library System တစ်ခုကို UML နဲ့ class diagram ဆွဲကြည့်ပါ။ တစ်ယောက် နဲ့ တစ်ယောက် တူမှာ မဟုတ်ပေမယ့် ကောင်းမွန်သည့် အလေ့အကျင့် တစ်ခု ရှိအောင် ဆွဲကြည့်ပါ။

ကျွန်တော် အကြမ်း ဆွဲထားသည့် design ကတော့ အောက်ပါ အတိုင်း ဖြစ်ပါတယ်။

